C语言二维数组详解

深入理解原理与应用，掌握多维数据处理

1. 二维数组的概念

二维数组是指同类型数据类型元素按行和列排列的**矩形**集合。可以把二维数组写成行和列的排列方式来理解二维数组的逻辑结构。

以同一维数组的基础上理解二维数组，可以理解为：二维数组是把一维数组作为数组的元素。

	0	1	2
0	1	2	3
1	4	5	6

二维数组的矩形图
示例数组可视化图解

简单数组示：

- 数组1数组0
- 数组2数组1

2. 二维数组的创建和初始化

语法定义：

```
type arr_name[行数][列数];
```

创建示例：

```
int arr[2][3];
```

表示定义了一个2行3列的数组。

完全初始化

```
#include<stdio.h>
int main()
{
    // 明示全数据的以列的方式顺序进行初始化
    int arr[2][3] = { 1,2,3,4,5,6 };
    return 0;
}
```

指定行列初始化

```
#include<stdio.h>
int main()
{
    int arr[2][3] = { {1,2,3}, {4,5,6} };
    int arr[2][3] = { {1,2}, {4} };
    return 0;
}
```

不完全初始化

```
#include<stdio.h>
int main()
{
    int arr[2][3] = { 1,2,3,4 };
    int arr[2][3] = { 0 };
    return 0;
}
```

注意

二维数组定义的时候可以省略行，不能省略列。（程序会根据列自动推导出行）

```
#include<stdio.h>
int main()
{
    int arr[3][] = { 1,2,3,4,5,6 };
    return 0;
}
```

3. 二维数组的使用

3.1 二维数组下标

	0	1	2
0	1	2	3
1	4	5	6

二维数组下标示意图
示例数组可视化图解

定义第二维数组如下：

```
#include<stdio.h>
int main()
{
    int arr[2][3] = { 1,2,3,4,5,6 };
    return 0;
}
```

二维数组访问规则

二维数组的访问先通过行的下标，找上图所示矩形表格的数组的位置，然后可以通过列的下标，行为0，列为0，即二维数组下标的工作，数组名称行列的方式。

访问示例：

- arr[0][0] 对应数组元素 1
- arr[0][1] 对应数组元素 2
- arr[1][2] 对应数组元素 6

3.2 通过数组下标访问元素

3.2.1 访问单维数组

```
#include<stdio.h>
int main()
{
    int arr[2][3] = { 1,2,3,4,5,6 };
    printf("%d\n", arr[0][2]);
    printf("%d\n", arr[1][0]);
    printf("%d\n", arr[1][2]);
    return 0;
}
```

3.2.2 访问数组全体的所有数据

```
#include<stdio.h>
int main()
{
    int arr[2][3] = { 1,2,3,4,5,6 };
    int row = 2;
    int col = 3;
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

3.2.3 访问部分人元素

```
#include<stdio.h>
int main()
{
    int arr[2][3];
    int row = 2;
    int col = 3;
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            scanf("%d", &arr[i][j]);
        }
    }
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

4. 二维数组在内存当中的存储

二维数组的内存特性

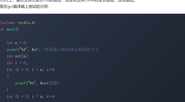
数组本身是一块连续的内存空间，所以无论是一维数组还是二维数组，我们通过打印地址可以发现内存地址是按顺序的。

```
#include<stdio.h>
int main()
{
    int arr[2][3] = 100;
    int row = 2;
    int col = 3;
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            printf("Char[%d][%d] = %p\n", i, j, &arr[i][j]);
        }
    }
    return 0;
}
```

```
Char[0][0] = 0027BFB0C0
Char[0][1] = 0027BFB0C4
Char[0][2] = 0027BFB0C8
Char[1][0] = 0027BFB0CA
Char[1][1] = 0027BFB0CE
Char[1][2] = 0027BFB0D2
```

整型的二维数组，每个元素4个字节，所以各地址之间间隔为4。

而一个数组的最当中相对（就是说）二维数组的第一维数组作为数组的元素，所以整个的内存地可以看做是数字的构成



二维数组在内存当中的图解
示例数组可视化图解

5. 二维数组习题练习

1. 编写一个程序，实现一个2×3矩阵的转置操作。矩阵转置是指将矩阵的行和列互换。原数组矩阵的行数和列数分别为原矩阵的列数和行数。

```
原矩阵为：
1 2 3
4 5 6

转置后的矩阵：
1 4
2 5
3 6
```

```
#include<stdio.h>
int main()
{
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    // 定义转置矩阵
    printf("原矩阵为：\n");
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    // 定义转置矩阵，行数和列数互换
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            // 交换行和列，将原矩阵的元素
            // 放到转置矩阵的对应位置
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }
    // 打印转置后的矩阵
    printf("\n转置后的矩阵：\n");
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

6. C99中的变长数组

在C99标准之前，C语言不支持数组的动态长度，数组大小的指定只能使用常量。常量表达式，或者枚举类型实现动态数组的，可以编译通过。

```
int arr[10];
int arr[sizeof(int)];
int arr[4] = {1,2,3,5};
```

这种固定长度数组，让我们在编译期就不知道大小，程序编译时大了直接空间，有时候程序会大了不能相信。

C99中有一个**变长数组（variable length array，简称 VLA）**的新特性，允许我们可以使用变量来指定数组大小，语法下面的代码：

```
int n = 5;
int arr[n];
```

上面代码中，数组arr数据变长数组，因为它的长度取决于变量n的值，编译期间定义事先确定，只有运行时才能知道n的值。

这个数组的特点和限制：数组长度在编译期间是一个固定的长度，程序可以运行时根据实际需求来决定。

- 编译器不会在编译期间，要求知道数组的具体长度，**而是在运行时根据变量来确定长度**。
- 在一个已经定义数组的函数，变长数组的长度**可以是任何整数表达式**，它可能是一个常量，也可能是变量，甚至是一个函数的返回值。
- 数组长度必须是一个整数，即不能是浮点数或其他类型。**数组长度一旦确定就不能再改变了。**

在VLA中，通常使用变量来定义数组的长度，例如使用变量n来指定数组的大小。

下面是在Linux编辑器上测试的示例：

```
#include<stdio.h>
int main()
{
    int n = 5;
    scanf("%d", &n);// 使用变量来指定数组的大小
    int arr[n];
    int i = 0;
    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

注意：

需要注意的是在VLA最上是无初始，即只可以通指定大小的数组。

三 本节总结

- 二维数组的概念和结构特性
- 二维数组的创建语法和多种初始化方法
- 二维数组下标的访问规则
- 二维数组元素的访问、输入和输出
- 二维数组在内存中的连续存储特性
- 矩阵转置数组实现实例
- C99标准中的变长数组（VLA）实现特性